

SPSS Modeling II

Elliot Dean

2025-06-09

Code translation by Gemini

Load Necessary Libraries

```
library(tidyverse)    # data wrangling & ggplot2
library(janitor)       # functions for examining and cleaning dirty data
library(haven)         # to read SPSS .sav files
library(caret)         # train/test split, cross-validation
library(broom)         # tidy model output
library(car)           # VIF, Durbin-Watson
library(lmtest)        # Breusch-Pagan test
library(ggpmisc)       # Automatically print the regression equation for ggplot
library(ggpubr)
# library(reticulate)
```

Importing and Organizing Data

```
df <- read_sav('notes/ECF_NE-Area_Data.sav')
# Convert to a plain data.frame or tibble if you prefer:
df <- as_tibble(df) |>
  clean_names()
```

Drop dupe columns + content

```
# Suppose df is your data.frame
# 1) Transpose to get columns as rows
mat <- t(as.matrix(df))

# 2) Identify duplicated rows (i.e., duplicate columns in original)
dups <- duplicated(mat)

# 3) Keep only non-duplicates
df <- df[ , !dups]

# Drop dupe column names
# df <- df[ , !duplicated(names(df))]
```

```
# 1. Initial Frequencies (SPSS: FREQUENCIES CCD. FREQUENCIES ECF.)
# These are typically for initial data exploration.
# In R:
# summary(df$CCD)
# summary(df$ECF)
# or for value counts:
# table(df$CCD)
# table(df$ECF)
```

Create Price Categories (SPSS: COMPUTE priceclass = 0. IF (saleprice LT ...) etc.)

```
df <- df |>
  mutate(
    priceclass = case_when(
      saleprice < 48000 ~ 1,
      saleprice >= 48000 & saleprice < 67000 ~ 2,
      saleprice >= 67000 & saleprice < 94000 ~ 3,
      saleprice >= 94000 & saleprice < 134000 ~ 4,
      saleprice >= 134000 ~ 5,
      TRUE ~ 0 # Default if none of the above conditions are met (similar to SPSS's initial C
OMPUTE = 0)
    )
  )

# Value Labels for priceclass (SPSS: VALUE LABELS priceclass ...)
df$priceclass <- factor(df$priceclass,
  levels = c(1, 2, 3, 4, 5),
  labels = c('Below 48000', '48000 to 67000', '67000 to 94000', '94000
to 134000', '134000 +'))

# Cross-tabulations (SPSS: CROSSTABS priceclass BY sRESB_STYLE.)
# In R: table(df$priceclass, df$sRESB_STYLE)
```

Create 'IN' field for filtering property styles (SPSS: COMPUTE IN=0. IF (Sresb_style='SINGLE FAMILY') IN=1. etc.)

```
df <- df |>
  mutate(
    instyle = case_when(
      s_resb_style == 'SINGLE FAMILY' ~ 1,
      s_resb_style == '1/2 DUPLEX' ~ 1,
      TRUE ~ 0 # Default to 0
    )
  )

# Select if IN=1 (SPSS: SELECT IF (IN=1).)
df_filtered_in <- df |>
  filter(instyle == 1)
```

Identify Duplicate Cases

```

# First, clean parcelno if it has special characters (SPSS: REPLACE '.' WITH 'P'; '-' WITH 'D'.)
df_filtered_in <- df_filtered_in |>
  mutate(
    parcelno = str_replace_all(parcelno, "\\.", "P"),
    parcelno = str_replace_all(parcelno, "-", "D")
  )

# Identify and keep the Last duplicate
df_unique_sales <- df_filtered_in |>
  arrange(parcelno, saleprice, saledate) |> # Sort first
  group_by(parcelno, saleprice, saledate) |>
  mutate(
    primary_first = row_number() == 1,
    primary_last = row_number() == n()
  ) |>
  ungroup() |>
  filter(primary_last == TRUE) # Keep only the last matching case

# You might want to remove the temporary PrimaryFirst and PrimaryLast columns if not needed
df_unique_sales <- df_unique_sales |>
  select(-primary_first, -primary_last)

# Value Labels for PrimaryLast (if you kept it for inspection)
# This part would typically be used before filtering out duplicates.
# df_filtered_in$PrimaryLast_Label <- factor(df_filtered_in$PrimaryLast, levels = c(0, 1), la
bels = c('Duplicate Case', 'Primary Case'))

```

Eliminate Transaction Where Price = Assessment (Non-Market) (SPSS: COMPUTE priceisasmt_ratio = avwhensold / saleprice. etc.)

```

df_processed <- df_unique_sales |>
  mutate(
    priceisasmt_ratio = avwhensold / saleprice,
    priceisasmt_ratio2 = priceisasmt_ratio,
    priceisasmt_ratio2 = ifelse(priceisasmt_ratio >= 0.90 & priceisasmt_ratio <= 1.1, 1, priceisasmt_ratio2),
    priceisasmt = ifelse(priceisasmt_ratio2 == 1, 0, 1) # 1 if not assessment price, 0 if it is
  )

# Select if priceisasmt=1 (SPSS: SELECT IF priceisasmt=1.)
df_processed <- df_processed |>
  filter(priceisasmt == 1)

```

Filter out bad data (SPSS: COMPUTE OUT=0. IF (s_totalsqft<400) OUT=1. etc.)

```
df_processed <- df_processed |>
  mutate(
    out = 0,
    out = ifelse(s_totalsqft < 400, 1, out),      # Filter out lots too small
    out = ifelse(s_resb_floorarea < 400, 1, out), # Filter out buildings too small
    out = ifelse(saleprice < 10000, 1, out)       # Filter out prices too low
    # Commented out original SPSS conditions:
    # out = ifelse(isVacantLandSale == 1, 1, out),
    # out = ifelse(LANDBANKSALES == 1, 1, out),
    # out = ifelse(priceisasmt_ratio == 0, 1, out)
  ) |>
  filter(out == 0) # Select if out = 0
```

Create Sale Price Per SF Field (SPSS: COMPUTE spps = saleprice/s_resb_floorarea.)

```
df_processed <- df_processed |>
  mutate(
    spps = saleprice / s_resb_floorarea
  )
```

Property Type Categorical Transformation (SPSS: COMPUTE onestory=0. IF (s_resb_styhgt LE 2) onestory = 1. etc.)

```
df_processed <- df_processed |>
  mutate(
    onestory = ifelse(s_resb_styhgt <= 2, 1, 0),
    twostory = ifelse(s_resb_styhgt > 2 & s_resb_styhgt <= 5, 1, 0),
    check     = onestory + twostory, # Sum of these two (SPSS: COMPUTE CHECK=SUM(onestory TO t
    wostory).)
    single_family1story = ifelse(s_resb_style == 'SINGLE FAMILY' & onestory == 1, 1, 0),
    half_duplex1story   = ifelse(s_resb_style == '1/2 DUPLEX' & onestory == 1, 1, 0),
    half_duplex2story   = ifelse(s_resb_style == '1/2 DUPLEX' & twostory == 1, 1, 0)
    # SPSS comments out SINGLE_FAMILY2STORY and others, so I'll follow that.
    # SINGLE_FAMILY2STORY = ifelse(s_resb_style == 'SINGLE FAMILY' & twostory == 1, 1, 0)
  )
```

Convert YearBuilt into TimePeriods (SPSS: COMPUTE erabuilt=0. IF (s_resb_yearbuilt LT 1910) erabuilt=1. etc.)

```

df_processed <- df_processed |>
  mutate(
    erabuilt = case_when(
      s_resb_yearbuilt < 1910 ~ 1,
      s_resb_yearbuilt >= 1910 & s_resb_yearbuilt < 1930 ~ 2,
      s_resb_yearbuilt >= 1930 & s_resb_yearbuilt < 1946 ~ 3,
      s_resb_yearbuilt >= 1946 & s_resb_yearbuilt < 1961 ~ 4,
      s_resb_yearbuilt >= 1961 & s_resb_yearbuilt <= 1990 ~ 5,
      s_resb_yearbuilt > 1990 ~ 6,
      TRUE ~ 0
    )
  )

# Value Labels for erabuilt
df_processed$erabuilt <- factor(df_processed$erabuilt,
                               levels = c(1, 2, 3, 4, 5, 6),
                               labels = c('Pre 1910', '1911-1929', '1930-1945', '1946-1960',
'1961-1990', 'Post 1990'))

# Recode erabuilt into binaries
df_processed <- df_processed |>
  mutate(
    # YB_PRE_1910 = ifelse(erabuilt == 1, 1, 0), # Commented out in SPSS
    yb_pre_1929 = ifelse(erabuilt %in% c('Pre 1910', '1911-1929'), 1, 0),
    # YB_1930_TO1945 = ifelse(erabuilt == 3, 1, 0), # Commented out in SPSS
    yb_1946_to_1960 = ifelse(erabuilt == '1946-1960', 1, 0),
    yb_post_1961 = ifelse(erabuilt %in% c('1961-1990', 'Post 1990'), 1, 0)
  )

```

Condition Code Categorical Transformation (SPSS: RECODE s_cond (2=1)(ELSE=0) INTO good. etc.)

```

df_processed <- df_processed |>
  mutate(
    # EXCELLENT = ifelse(s_cond == 0, 1, 0), # Commented out in SPSS
    # VERY_good = ifelse(s_cond == 1, 1, 0), # Commented out in SPSS
    good = ifelse(s_cond == 2, 1, 0),
    # AVG = ifelse(s_cond == 3, 1, 0), # Commented out in SPSS
    fair = ifelse(s_cond == 4, 1, 0),
    poor = ifelse(s_cond == 5, 1, 0),
    very_poor_unsound = ifelse(s_cond %in% c(6, 7), 1, 0)
    # UNSOUND = ifelse(s_cond == 7, 1, 0) # Commented out in SPSS
  )

```

Basement, Fireplace, Bathrooms

```
# 11. Bathrooms (SPSS: RECODE s_resb_fullbaths (2 3 ... =1) INTO two_plus_baths. etc.)
df_processed <- df_processed |>
  mutate(
    two_plus_baths = ifelse(s_resb_fullbaths >= 2 & s_resb_fullbaths <= 10, 1, 0),
    powder_room    = ifelse(s_resb_halfbaths >= 1 & s_resb_halfbaths <= 5, 1, 0)
  )

# 12. Fireplaces (SPSS: RECODE s_resb_fireplace(1 2 ... =1) INTO FIREPLACE. etc.)
df_processed <- df_processed |>
  mutate(
    fireplace = ifelse(s_resb_fireplace >= 1 & s_resb_fireplace <= 6, 1, 0)
  )

# 13. Basement Transformations (SPSS: COMPUTE pctcrawl=(s_resb_crawspace/s_resb_groundarea).
etc.)
df_processed <- df_processed |>
  mutate(
    pctcrawl = s_resb_crawspace / s_resb_groundarea,
    pctslab  = s_resb_slabarea / s_resb_groundarea,
    pctbsmnt = s_resb_basementarea / s_resb_groundarea
  ) |>
  # Handle potential division by zero if s_resb_groundarea is 0, setting PCT to 0 or NA
  # For simplicity here, assuming s_resb_groundarea is never 0 or handling it beforehand.
  mutate(
    crawl = ifelse(pctcrawl > pctbsmnt & pctcrawl > pctslab, 1, 0),
    slab  = ifelse(pctslab > pctcrawl & pctslab > pctbsmnt, 1, 0)
  )
```

Quality, Garages

```

# 14. Quality Grade Transformations (SPSS: RECODE s_resb_bldgclass (3 4 5=1)(ELSE=0) INTO qab
ove_avg. etc.)
df_processed <- df_processed |>
  mutate(
    # QBEST = ifelse(s_resb_bldgclass == 5, 1, 0), # Commented out in SPSS
    qabove_avg = ifelse(s_resb_bldgclass %in% c(3, 4, 5), 1, 0),
    qbelow_avg = ifelse(s_resb_bldgclass %in% c(0, 1), 1, 0)
    # Qpoor = ifelse(s_resb_bldgclass == 0, 1, 0) # Commented out in SPSS
  )

# 15. Garages (SPSS: COMPUTE GARAGE=0. IF (SRESB_GARAGEAREA GE 200) GARAGE=1. etc.)
df_processed <- df_processed |>
  mutate(
    garage = ifelse(s_resb_garagearea >= 200, 1, 0)
  ) |>
  mutate(
    one_car_garage = ifelse(s_reb_gartype == 1, 1, 0),
    two_car_garage = ifelse(s_reb_gartype == 2, 1, 0)
  ) |>
  mutate( # Garage spaces categories
    garagespaces = case_when(
      s_resb_garagearea > 200 & s_resb_garagearea < 420 ~ 1,
      s_resb_garagearea >= 420 & s_resb_garagearea < 600 ~ 2, # Note: SPSS had GT 419, which
is >= 420 for integers
      s_resb_garagearea >= 600 & s_resb_garagearea < 820 ~ 3, # Note: SPSS had GT 599, which
is >= 600 for integers
      s_resb_garagearea >= 820 ~ 4, # Note: SPSS had GT 819, which is >= 820 for integers
      TRUE ~ 0 # Default if no category matches
    )
  ) |>
  mutate(
    ln_garagespaces = log(garagespaces) # Natural Logarithm, similar to SPSS LN()
  )
# Note: LN(0) will produce -Inf. You might need to handle cases where GARAGESPACES is 0.
# A common approach is to add a small constant or set to 0/NA if GARAGESPACES is 0.

```

Street Class, Heating / Cooling, Size Categories

```

# 16. Street Class (SPSS: RECODE RD_CLASS ('A21' 'A31' =1)(ELSE=0) INTO ARTERIAL. etc.)
df_processed <- df_processed |>
  mutate(
    arterial = ifelse(rd_class %in% c('A21', 'A31'), 1, 0)
  )

# 17. Heating Cooling (SPSS: RECODE s_resb_heat (2=1)(ELSE=0) INTO HOT_WATER. etc.)
df_processed <- df_processed |>
  mutate(
    # FORCED_AIR = ifelse(s_resb_heat %in% c(0, 1, 9), 1, 0), # Commented out in SPSS
    hot_water      = ifelse(s_resb_heat == 2, 1, 0),
    elec_baseboard = ifelse(s_resb_heat == 3, 1, 0),
    # ELEC_RADIANT = ifelse(s_resb_heat == 4, 1, 0), # Commented out in SPSS
    # RADIANT_FLOOR = ifelse(s_resb_heat == 5, 1, 0), # Commented out in SPSS
    # ELEC_WALL = ifelse(s_resb_heat == 6, 1, 0), # Commented out in SPSS
    # SPACE_HEATER = ifelse(s_resb_heat == 7, 1, 0), # Commented out in SPSS
    wall           = ifelse(s_resb_heat %in% c(6, 8), 1, 0),
    central_air    = ifelse(s_resb_heat == 9, 1, 0)
  )

# 18. Building Size Categories (SPSS: COMPUTE SIZE = 0. IF (s_resb_floorarea LT 830) SIZE=1.
etc.)
df_processed <- df_processed |>
  mutate(
    size = case_when(
      s_resb_floorarea < 830 ~ 1,
      s_resb_floorarea >= 830 & s_resb_floorarea < 1018 ~ 2,
      s_resb_floorarea >= 1018 & s_resb_floorarea < 1255 ~ 3,
      s_resb_floorarea >= 1255 & s_resb_floorarea < 1632 ~ 4,
      s_resb_floorarea >= 1632 ~ 5,
      TRUE ~ 0
    )
  ) |>
  mutate( # Binaries for SIZE
    smallest = ifelse(size == 1, 1, 0),
    small    = ifelse(size == 2, 1, 0),
    # AVG = ifelse(SIZE == 3, 1, 0), # Commented out in SPSS
    large    = ifelse(size == 4, 1, 0),
    largest  = ifelse(size == 5, 1, 0)
  )

```

Time Analyses (SPSS: COMPUTE smonth = XDATE.MONTH(saledate). etc.) (DATE FORMAT NECESSARY)


```

df_processed <- df_processed |>
  mutate(saledate = ymd(saledate)) # Assuming YYYY-MM-DD or similar, adjust format if needed

df_processed <- df_processed |>
  mutate(
    smonth = month(saledate),
    syear = year(saledate)
  ) |>
  mutate( # Calculate months
    months = case_when(
      syear == 2022 ~ smonth - 5,
      syear == 2023 ~ smonth + 7,
      syear == 2024 ~ smonth + 19,
      TRUE ~ NA_real_ # Handle other years if any
    )
  ) |>
  mutate( # Spline variables
    months_1to9 = ifelse(months > 9, 9, months),
    months_10to16 = ifelse(months > 9, months - 9, 0),
    months_10to16 = ifelse(months_10to16 > 7, 7, months_10to16), # Cap at 7
    months_17to24 = ifelse(months > 16, months - 16, 0)
  ) |>
  mutate(
    ln_sppsf = log(sppsf) # Natural Logarithm
  )

# SPSS Regression Coefficients for Time Variables (Hardcoded rates)
# In R, you'd usually get these from a model output, but here they are used for calculation.
RATE1 <- 0
RATE2 <- 0.008022
RATE3 <- 0.015017 # From SPSS: COMPUTE RATE3 = 0.

df_processed <- df_processed |>
  mutate(
    price_index = (1 + RATE1)^months_1to9 * (1 + RATE2)^months_10to16 * (1 + RATE3)^months_17
to24
  )

# Time Adjustment Factors (taf) (SPSS: COMPUTE END_INDEX=.10709. COMPUTE taf=END_INDEX/price_
index. etc.)
END_INDEX <- 0.10709
df_processed <- df_processed |>
  mutate(
    taf = end_index / price_index,
    tasp = saleprice * taf,
    taspsf = tasp / s_resb_floorarea
  )

```

SBLDSF Transformations, Lot Size, ECF

```

# 20. SBLDSF Transformations (SPSS: COMPUTE lnsbldsf= LN(s_resb_floorarea). etc.)
# Calculate medians for different styles first
# SPSS code for medians
median_single_family_floorarea <- df_processed |>
  filter(s_resb_style == 'SINGLE FAMILY') |>
  pull(s_resb_floorarea) |>
  median(na.rm = TRUE) # Output from SPSS: 1056

median_half_duplex_floorarea <- df_processed |>
  filter(s_resb_style == '1/2 DUPLEX') |>
  pull(s_resb_floorarea) |>
  median(na.rm = TRUE) # Output from SPSS: 840

# In SPSS, they hardcode these medians into a RECODE, I'll use `case_when` for `base_bldgsize`
df_processed <- df_processed |>
  mutate(
    lnsbldsf      = log(s_resb_floorarea),
    sbldsfint     = s_resb_floorarea - 432, # 432 is smallest building in master file
    lnsbldsfint   = log(s_resb_floorarea - 432),
    sbldsfmed     = s_resb_floorarea - 1051 # 1051 is median
    # lnsbldsfMED = log(s_resb_floorarea - 1051) # This would require handling negative numbers
  ) |>
  # Using the medians calculated above to create base_bldgsize and related ratios
  mutate(
    base_bldgsize = case_when(
      s_resb_style == 'SINGLE FAMILY' ~ median_single_family_floorarea, # 1056
      s_resb_style == '1/2 DUPLEX' ~ median_half_duplex_floorarea, # 840
      TRUE ~ NA_real_ # Handle other styles if any
    )
  ) |>
  mutate(
    base_bldgsize_ratio      = s_resb_floorarea / base_bldgsize,
    ln_base_bldgsize_ratio   = log(base_bldgsize_ratio),
    ln_sbldsf_bsz_single     = ifelse(s_resb_style == 'SINGLE FAMILY', ln_base_bldgsize_ratio,
    , 0),
    ln_sbldsf_bsz_halfduplex = ifelse(s_resb_style == '1/2 DUPLEX', ln_base_bldgsize_ratio,
    0)
  )
# Note: Handle potential -Inf from log() if s_resb_floorarea - 432 <= 0, or if base_bldgsize_ratio <= 0.

# 21. Lot Size (SPSS: RECODE SRESB_STYLE ('SINGLE FAMILY' =4599)('1/2 DUPLEX' =3180) INTO BASE_LOTSIZE. etc.)
# Calculate medians for different styles (SPSS output provided: Single Family 4599, 1/2 Duplex 3180)
median_single_family_lotsize <- 4599
median_half_duplex_lotsize   <- 3180

df_processed <- df_processed |>

```

```

mutate(
  base_lotsize = case_when(
    s_resb_style == 'SINGLE FAMILY' ~ median_single_family_lotsize,
    s_resb_style == '1/2 DUPLEX' ~ median_half_duplex_lotsize,
    TRUE ~ NA_real_ # Handle other styles
  )
) |>
mutate(
  base_lotsize_ratio = s_totalsqft / base_lotsize,
  ln_base_lotsize_ratio = log(base_lotsize_ratio),
  landratio = s_totalsqft / base_lotsize, # Redundant, but matches SPSS
  landratio2 = landratio,
  landratio2 = ifelse(landratio >= 0.90 & landratio <= 1.10, 1, landratio2),
  landratio2 = ifelse(s_resb_style != 'SINGLE FAMILY' & landratio > 3, 3, landra
tio2), # Cap non-single family
  landratio2 = ifelse(s_resb_style == 'SINGLE FAMILY' & landratio > 4, 4, landra
tio2), # Cap single family
  ln_landratio = log(landratio2)
)
# Again, watch out for log(0) or log(negative).

# 22. ECFs (Recoding ECFs into binary variables)
# This section creates binary indicators for various ECF (Economic Factor) codes.
# This will be extensive if you need all of them, but the pattern is clear.
# Example:
df_processed <- df_processed |>
mutate(
  # ecf2R211 = ifelse(ecf == '2R211', 1, 0), # Commented out in SPSS
  ecf2r218_539 = ifelse(ecftbl %in% c('2R218', '5R539'), 1, 0),
  ecf3r301_302 = ifelse(ecftbl %in% c('3R301', '3R302'), 1, 0),
  # ecf3R302 = ifelse(ecf == '3R302', 1, 0), # Commented out in SPSS
  # ecf3R303 = ifelse(ecf == '3R303', 1, 0), # Commented out in SPSS
  # ecf3R304 = ifelse(ecf == '3R304', 1, 0), # Commented out in SPSS
  # ecf3R305 = ifelse(ecf == '3R305', 1, 0), # Commented out in SPSS
  ecf3r306 = ifelse(ecftbl == '3R306', 1, 0),
  ecf3r307 = ifelse(ecftbl == '3R307', 1, 0),
  ecf3r308 = ifelse(ecftbl == '3R308', 1, 0),
  ecf3r309 = ifelse(ecftbl == '3R309', 1, 0),
  ecf3r310 = ifelse(ecftbl == '3R310', 1, 0),
  ecf3r311 = ifelse(ecftbl == '3R311', 1, 0),
  ecf3r312 = ifelse(ecftbl == '3R312', 1, 0),
  ecf3r313 = ifelse(ecftbl == '3R313', 1, 0),
  ecf3r314 = ifelse(ecftbl == '3R314', 1, 0),
  ecf3r315 = ifelse(ecftbl == '3R315', 1, 0),
  ecf3r316 = ifelse(ecftbl == '3R316', 1, 0),
  ecf3r317 = ifelse(ecftbl == '3R317', 1, 0),
  ecf3r318 = ifelse(ecftbl == '3R318', 1, 0),
  ecf3r319 = ifelse(ecftbl == '3R319', 1, 0),
  ecf3r320 = ifelse(ecftbl == '3R320', 1, 0),
  ecf3r321 = ifelse(ecftbl == '3R321', 1, 0)
  # ecf3R322 = ifelse(ecf == '3R322', 1, 0) # Partial line in SPSS

```

```
)
```

Save dataframes

```
# The save function is included in base R
save(df, df_filtered_in, df_processed, df_unique_sales, file = 'data/spss_dfs.RData')
```

Check cols

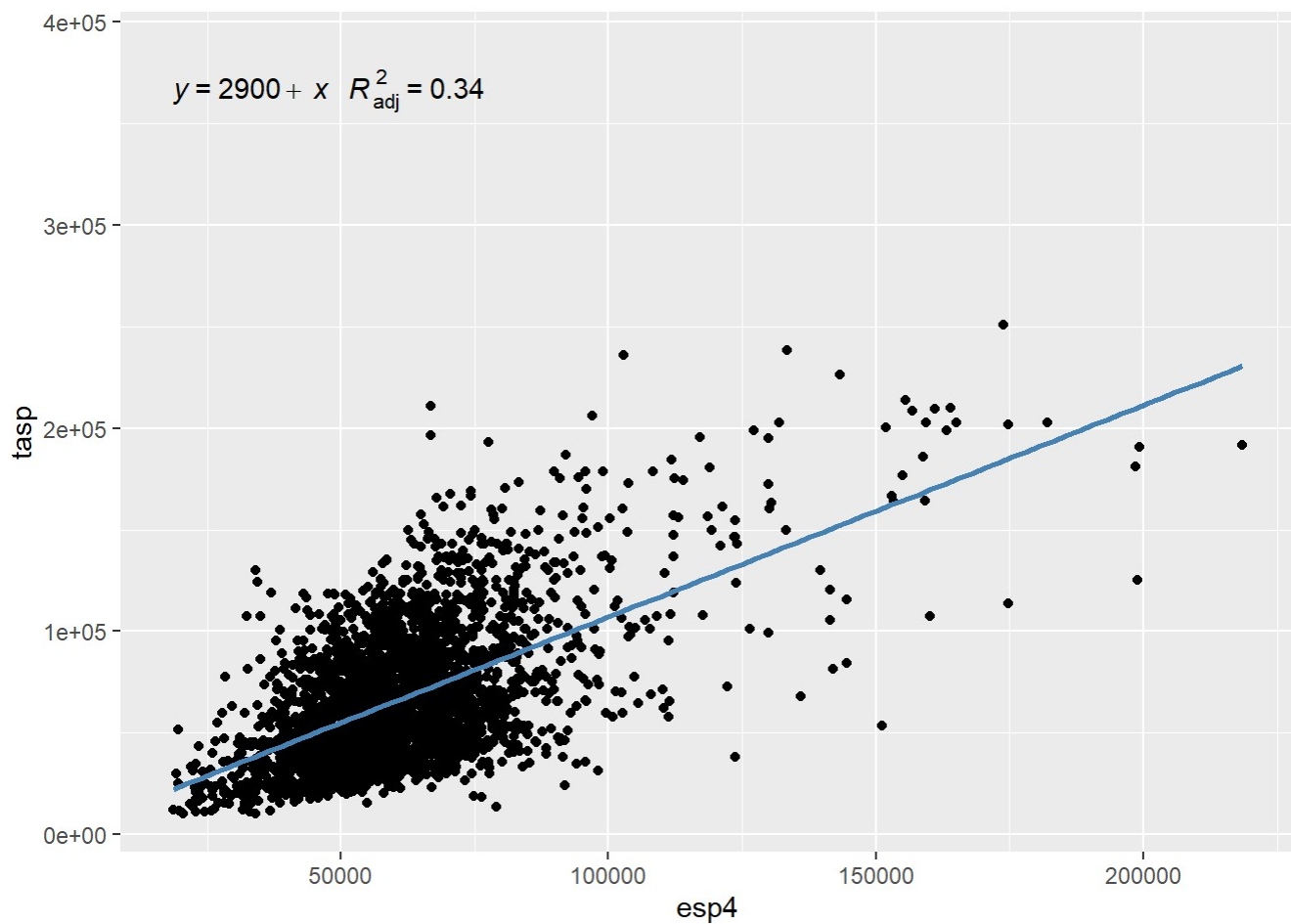
```
# Suppose your model formula is:  $y \sim x1 + x2 + x3$ 
# and your data frame is 'mydf'.

# Check for any NA/NaN/Inf in those predictor columns:
lapply(mydf[c("ln_tasp", "ln_landratio", "x3")], function(col) {
  any(is.na(col)) || any(is.nan(col)) || any(is.infinite(col))
})
```

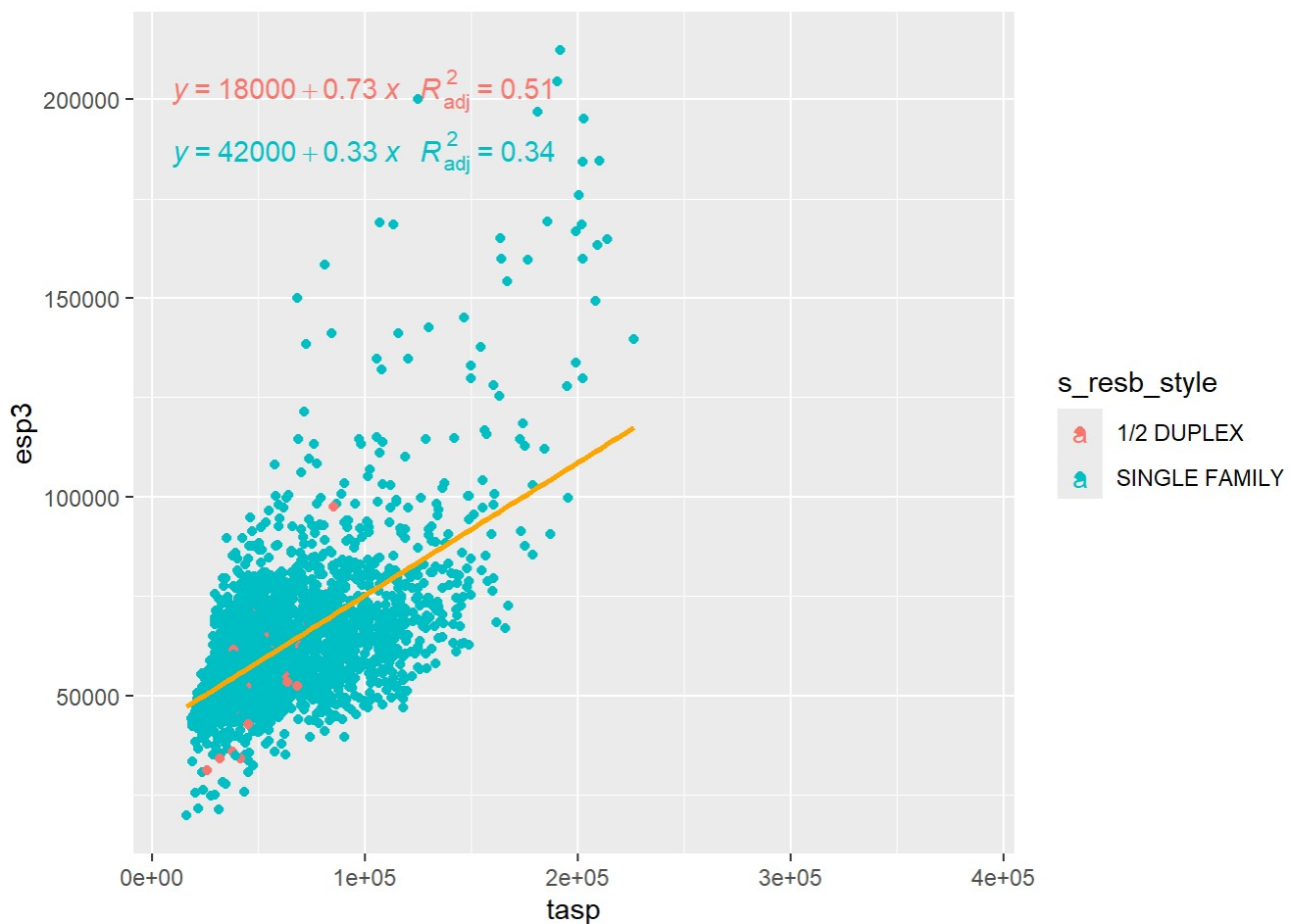
```
dfff <- df_processed |>
  drop_na()
```

Visuals

```
# esp1, esp2, esp3, esp4
# ratio1, ratio2, ratio3, ratio4
ggplot(df, aes(x = esp4, y = tasp)) +
  geom_point() +
  stat_smooth(method = "lm", se = FALSE, color = "steelblue") +
  stat_regline_equation(
    aes(label = paste(after_stat(eq.label), after_stat(adj.rr.label), sep = "~~~")),
    label.x.npc = "left", # use "npc" coordinates 0-1 in panel
    label.y.npc = "top"
  )
```



```
ggplot(df, aes(x = tasps, y = esp3, colour = s_resb_style)) +
  geom_point() +
  stat_smooth(method = "lm", se = FALSE, color = "orange") +
  stat_regline_equation(
    aes(label = paste(after_stat(eq.label), after_stat(adj.rr.label), sep = "~~~")),
    label.x.npc = "left",  # use "npc" coordinates 0-1 in panel
    label.y.npc = "top"
  )
```



Regression Models

```
# 23. Regression Models (SPSS: REGRESSION /DEPENDENT=LN_tasp /METHOD=BACKWARD ...)
# The main regression model:
# Note: Ensure all variables used in the model exist in your 'df_processed' data frame.
# If some variables are not created due to previous comments in SPSS, you'll need to adjust.
# Also, R's lm() doesn't directly support backward selection as an argument.
# You'd typically use step() for stepwise selection after an initial model, or specify variables manually.

# The second regression in the file (SPSS: REGRESSION /DEPENDENT=LN_tasp /METHOD=BACKWARD ...)
# This model uses the time-adjusted sale price (LN_tasp).
# lnsblsfINT was part of the original SPSS REGRESSION

lm_model_tasp <- lm(ln_tasp ~ ln_landratio + lnsblsfint + arterial +
  single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  yb_pre_1929 + yb_1946_to_1960 + yb_post_1961 + crawl + slab +
  ln_garagespaces + fireplace + hot_water + elec_baseboard + wall +
  central_air + months_1to9 + months_10to16 + months_17to24,
  data = dfff)
summary(lm_model_tasp)
```

```
##
## Call:
## lm(formula = ln_tasp ~ ln_landratio + lnsblsdfint + arterial +
##     single_family1story + half_duplex1story + half_duplex2story +
##     smallest + small + large + largest + two_plus_baths + powder_room +
##     good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
##     yb_pre_1929 + yb_1946_to_1960 + yb_post_1961 + crawl + slab +
##     ln_garagespaces + fireplace + hot_water + elec_baseboard +
##     wall + central_air + months_1to9 + months_10to16 + months_17to24,
##     data = dfff)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-1.28005	-0.29167	0.01093	0.32689	1.01566

```
##
## Coefficients: (6 not defined because of singularities)
##
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      9.720411    2.184364   4.450 1.15e-05 ***
## ln_landratio      0.013619    0.136538   0.100  0.92060
## lnsblsdfint       0.191934    0.333174   0.576  0.56493
## arterial         -0.074616    0.100612  -0.742  0.45880
## single_family1story 0.019400    0.062093   0.312  0.75489
## half_duplex1story      NA          NA        NA      NA
## half_duplex2story      NA          NA        NA      NA
## smallest            NA          NA        NA      NA
## small              NA          NA        NA      NA
## large              0.070033    0.100767   0.695  0.48750
## largest            0.206385    0.206209   1.001  0.31758
## two_plus_baths       0.219245    0.329271   0.666  0.50594
## powder_room        -0.019677    0.136879  -0.144  0.88578
## good               0.461691    0.323906   1.425  0.15492
## fair              -0.073205    0.060449  -1.211  0.22669
## poor              -0.368178    0.188576  -1.952  0.05167 .
## very_poor_unsound    0.386463    0.328141   1.178  0.23969
## qabove_avg          0.749203    0.479832   1.561  0.11932
## qbelow_avg         -0.025765    0.059805  -0.431  0.66687
## yb_pre_1929        -0.100946    0.085583  -1.180  0.23898
## yb_1946_to_1960     0.066018    0.057933   1.140  0.25524
## yb_post_1961        0.109745    0.272845   0.402  0.68776
## crawl             -0.346096    0.275335  -1.257  0.20958
## slab               0.536600    0.267891   2.003  0.04593 *
## ln_garagespaces     -0.092998    0.082646  -1.125  0.26124
## fireplace          -0.001771    0.056872  -0.031  0.97518
## hot_water           0.161605    0.099273   1.628  0.10443
## elec_baseboard      NA          NA        NA      NA
## wall               NA          NA        NA      NA
## central_air         0.073707    0.058796   1.254  0.21080
## months_1to9         0.022875    0.012727   1.797  0.07313 .
## months_10to16       0.019629    0.014997   1.309  0.19141
## months_17to24      -0.053898    0.014484  -3.721  0.00023 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4499 on 357 degrees of freedom
## Multiple R-squared:  0.1549, Adjusted R-squared:  0.09331
## F-statistic: 2.516 on 26 and 357 DF,  p-value: 8.853e-05
```

```
# For backward selection in R, you would typically use `step()`:
# # Define the formula for the full model
# formula_full_tasp <- as.formula(
#   "LN_tasp ~ LN_LANDRATIO + lnsbldsfINT + ARTERIAL +
#     single_family1story + half_duplex1story + half_duplex2story +
#     smallest + small + large + largest +
#     two_plus_baths + powder_room +
#     good + fair + poor + very_poor_unsound +
#     qabove_avg + qbelow_avg +
#     yb_pre_1929 + yb_1946_to_1960 + yb_post_1961 +
#     CRAWL + SLAB + LN_GARAGESPACES + FIREPLACE +
#     HOT_WATER + ELEC_BASEBOARD + WALL + CENTRAL_AIR +
#     months_1to9 + months_10to16 + months_17to24"
# )
#
# initial_lm_model_tasp <- lm(formula_full_tasp, data = df_processed)
# backward_lm_model_tasp <- step(initial_lm_model_tasp, direction = "backward", trace = FALSE)
# # Set trace=TRUE to see steps
# summary(backward_lm_model_tasp)

# Final descriptive statistics (SPSS: DES saleprice tasps taf taspps)
# In R:
# summary(df_processed$saleprice)
# summary(df_processed$tasp)
# summary(df_processed$taf)
# summary(df_processed$taspps)
```